Go言語を対象とした複数のGround Truth定義によるSBOM生成ツール精度比較

安岡 瑞希[1,a)]　小津 喬[2,b)]　中村 徹[1,c)]

概要：ソフトウェアサプライチェーン攻撃への対策として，ソフトウェアの部品一覧である Software Bill of Materials（SBOM）が注目されている．SBOM の人手による作成は現実的でないため，SBOM 生成ツールで自動生成される．生成ツールの正確性は，ツールが出力した SBOM を正解データである Ground Truth（GT）と比較することで評価される．しかし著者らの知る限り，従来の評価はいずれも単一の GT 定義に基づいており，実在する Go プロジェクトを対象とした GT に基づく評価は行われていない．本研究では，正解とみなす依存関係の範囲が異なる 3 通りの GT を定義し，1,528 件の Go プロジェクトを対象に，4 種類の SBOM 生成ツールの性能を評価した．その結果，評価結果が GT 定義の違いによって変動することを示した．たとえば Syft と cdxgen の F1 スコアは，GT-all ではそれぞれ 73.0%と 52.8%であったのに対し，GT-imported では 67.0%と 93.2%となり，優劣が逆転した．さらに，この逆転が，各ツールが依存関係を抽出する情報源の違いによって説明できることを示した．

キーワード：SBOM，ソフトウェアサプライチェーン，Go 言語，精度評価，Ground Truth

Accuracy Comparison of SBOM Generation Tools for Go under Multiple Ground Truth Definitions

Mizuki Yasuoka[1,a)]　Takashi Ozu[2,b)]　Toru Nakamura[1,c)]

Abstract: Software Bills of Materials (SBOMs), inventories of software components, help mitigate software supply chain attacks. Because manually creating SBOMs is impractical, automated SBOM generation tools are used, making their accuracy important. Accuracy is evaluated by comparing a generated SBOM with a ground truth (GT). However, to the best of our knowledge, previous evaluations use a single GT definition, and no GT-based evaluation has examined real-world Go projects. We define three GTs covering different dependency scopes and evaluate four SBOM generation tools on 1,528 Go projects. The results show that tool performance varies with the GT definition. For example, the F1 scores of Syft and cdxgen were 73.0% and 52.8% under GT-all, but 67.0% and 93.2% under GT-imported, reversing their ranking. We further show that this reversal can be explained by differences in the information sources from which the tools extract dependencies.

Keywords: SBOM, software supply chain, Go language, accuracy evaluation, ground truth

1. はじめに

ソフトウェアサプライチェーン攻撃は，ソフトウェアの開発や配布の過程に介入し，正規のソフトウェアに不正なコードを混入させる攻撃である．近年こうした攻撃が急増しており，その対策として，ソフトウェアを構成する依

[1] 東京都市大学 情報工学部
Department of Computer Science, Tokyo City University
[2] 国際電気通信基礎技術研究所（ATR）
[a)] g2331101@tcu.ac.jp
[b)] ozu@atr.jp
[c)] tonakamu@tcu.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

存ライブラリ等を列挙した一覧である **Software Bill of Materials（SBOM）** が注目されている．その整備は各国で政策的に進められており，米国では大統領令 EO 14028 が連邦政府調達においてソフトウェアサプライチェーンの整合性強化を求め [14]，国内では経済産業省が SBOM 導入の手引を策定及び公表している [15].

SBOM は脆弱性対応やライセンス調査に用いられるため，一覧が不正確であれば，使用されている依存の見落としや，使用されていない依存の過剰な報告を招く恐れがある．ツールが生成する SBOM の正確性は，正解一覧である **Ground Truth**（以下 GT）と突き合わせて評価する．しかし，GT の定義次第で正解の範囲は変わるため，評価結果そのものが変わる可能性がある．従来研究 [1], [2], [3], [4] では，Python や Java 等での GT ベースの評価が行われてきたが，いずれも単一の GT 定義に基づいており，また著者らの知る限り，実在の Go プロジェクトを対象とした GT ベースの評価は行われていない．

本稿の貢献は次の 3 点である．第一に，実在の Go プロジェクト 1,528 件を対象に，4 つの SBOM 生成ツールの精度を GT に基づいて比較した．第二に，GT を依存範囲の異なる 3 つの定義で比較し，定義によってツールの優劣が逆転することを定量的に示した．第三に，その逆転の要因を，各ツールが依存を読み取る情報源の違いから説明した．

本論文の構成は以下のとおりである．第 2 章では，Go における依存関係の単位と依存管理ファイルについて説明し，第 3 章では関連研究を示す．第 4 章では，依存範囲の異なる 3 つの GT 定義とリサーチクエスチョンを示し，第 5 章では評価手法を述べる．第 6 章では評価結果を示し，第 7 章ではツールの優劣が逆転する理由，実用上の示唆，および本研究の限界を考察する．第 8 章では本研究をまとめる．

2. 準備

本章では，Go における依存関係の単位と，go.mod および go.sum の役割について述べる．

2.1 Go の依存の単位

Go では，依存関係は**モジュール**が**パッケージ**を束ねる階層構造をなす．モジュールとは，go.mod によって定義され，1 つ以上のパッケージをまとめた依存管理の単位であり，バージョンはモジュール全体に対して付けられる．パッケージとは，同じディレクトリに置かれた .go ファイルのまとまりであり，import やコンパイルの単位となる [8]．SBOM や GT では，パッケージではなく，そのパッケージが属するモジュールを依存の単位として扱う．コードが他のパッケージを import して利用するとき，コードの import から静的に到達可能な依存が，後述する GT-imported に対応する．なお標準ライブラリは依存に数えず，外部から取得するパッケージが属するモジュールを SBOM の対象とする．また，Go では，_test.go で終わるファイルにテストコードを記述する．本研究では，通常のコードでは使われず，テストコードからのみ静的に到達可能な外部モジュールをテスト依存と呼ぶ．

2.2 go.mod と go.sum

go.mod は，使用する依存とそのバージョンをモジュール単位で記述する [8]．直接依存と間接依存の両方が記録される．go.mod は `go mod init` で初期化し，`go mod tidy` によって，プロジェクト全体で必要な依存が記録された状態に整えられる．`go mod tidy` は，通常のコードだけでなく，テストコード，他 OS 向けのコード，開発用のツール，および特定の条件でのみ使用されるコードで使われる依存も，必要な依存として go.mod に残す [8]．例えば，Windows 向けのコードでのみ使われる依存は，Linux 向けのビルドでは使われないが，go.mod には記録される．

go.sum は，直接依存や間接依存として取得したモジュールの内容が変更されていないことを確認するためのハッシュを記録する [8]．また，Go は，使用するバージョンを決めるために，各依存モジュールの go.mod に記載された依存とそのバージョンを確認する．その際に取得した go.mod についても，同じ目的でそのハッシュが go.sum に記録される [8]．

3. 関連研究

GT ベースの SBOM 精度評価は複数の言語で行われてきたが，正解（GT）の作り方は言語・研究ごとに異なる（**表 1**）．いずれの研究も単一の GT 定義に基づいており，GT 定義の選択が評価結果に与える影響は検討されていない．Go については，Reichmann ら [6] が，自作の検証用プロジェクトを 6 言語分用意し，5 つの SBOM 生成ツールが部品を検出できるかを調べている．ただし対象は実在のプロジェクトではなく，precision や recall によるツール間の比較も行っていない．したがって，著者らの知る限り，実在の Go プロジェクトを対象とした GT ベース評価は行われていない．本研究は，実在の Go プロジェクトを対象に GT ベースの評価を行い，かつ GT 定義を 3 通りに変えて結果の変化を比較する点に新規性がある．次章では，本研究で用いる 3 つの GT 定義を述べる．

4. 依存範囲に基づく Ground Truth の 3 定義

本章では，本研究で用いる依存範囲の異なる 3 つの GT の定義と包含関係，およびリサーチクエスチョンについて述べる．

4.1 定義と包含関係

正解データ GT を**表 2** に示す **GT-imported**，**GT-**

© 2026 Information Processing Society of Japan

表 1　各言語における既存の GT ベース評価と本研究の位置づけ
Table 1　Existing GT-based evaluations per language and our positioning.

言語	研究	正解（GT）の作り方
Python	Yu et al. (2024)[1]	導入実体
Python	Reality Check (2026)[4]	ロックファイル
Java	Balliu et al. (2023)[2]	Maven 依存ツリー
JavaScript	Rabbi et al. (2024)[3]	`npm list`
Rust	Reality Check (2026)[4]	ロックファイル
Ruby	Reality Check (2026)[4]	ロックファイル
PHP	Reality Check (2026)[4]	ロックファイル
C / C++	Tang et al. (2022)[5]	人手でコード読解
Go	Reichmann et al. (2026)[6]	自作パッケージ
Go	本研究	`go list` import 依存

表 2　正解データ（GT）の 3 つの定義
Table 2　Three definitions of the ground truth.

GT 定義	生成コマンド（要点）
GT-imported	`go list -deps -e ./...`
GT-imported+test	`go list -deps -test -e ./...`
GT-all	`go list -m -e all`

imported+test，**GT-all** の 3 通りで定義する．GT-imported は，実際にコードで使用される依存だけを集めたもので，テスト依存は含まれない．GT-imported+test は，GT-imported に test 専用依存を加えたものである．GT-all は，直接，間接，テスト，未使用のすべてを含み，最も広い定義である．ここで未使用とは，Linux 向けの通常コードおよびテストコードから静的に到達できない依存を指す．これら 3 定義は**図 1** に示すように，定義上，GT-imported ⊆ GT-imported+test ⊆ GT-all の包含関係にある．いずれも対象リポジトリのルートディレクトリで `go list`[7] を実行して生成する．GT-imported と GT-imported+test では，`GOOS=linux` を指定した．出力の整形を含む完全なコマンドは，付録 A.4 に示す．

4.2　研究目的とリサーチクエスチョン

本研究の主目的は，Go プロジェクトを対象に，GT ベースで 4 ツールの精度を評価することである．リサーチクエスチョンは次の 2 点である．

RQ1： GT 定義（GT-imported / GT-imported+test / GT-all）を変えると，SBOM 生成ツールの優劣は変わるか．

RQ2： RQ1 で優劣が変わるとすれば，その原因は何か．

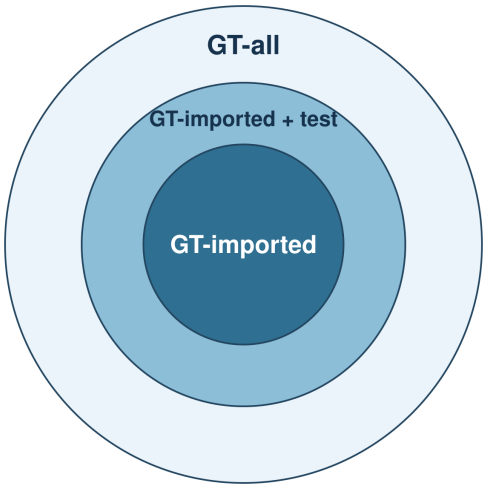


図 1　3 つの GT 定義の包含関係
Fig. 1　Inclusion relation among the three GT definitions.

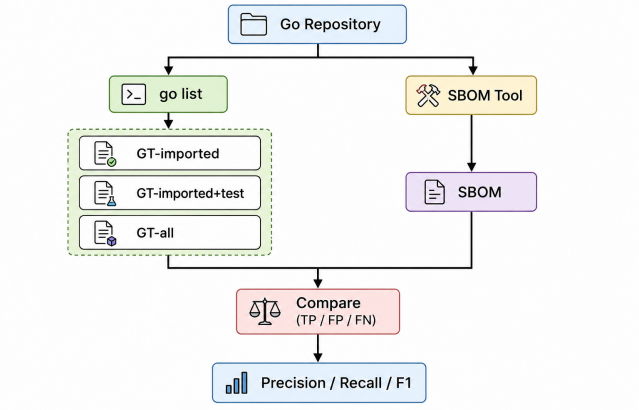


図 2　評価の流れ
Fig. 2　Overview of the evaluation process.

5.　評価手法

本章では，評価の流れ，対象データセット，評価対象ツールと実行環境，各ツールが依存を読み取る情報源，および評価指標について述べる．

5.1　評価の流れ

本研究の評価の流れを**図 2** に示す．各リポジトリから，`go list` による正解（GT）と各ツールによる SBOM を生成し，両者を突き合わせて精度を算出する．

5.2　対象データセット

評価対象は，Go のライブラリ集 awesome-go[9] に収録された 2,723 件のリポジトリである．このうち，取得に失敗した 13 件と，評価対象から除外した 1,182 件を除き，1,528 件を評価した．除外した 1,182 件は，`go.mod` を持たない 505 件と，GT-imported で外部依存が検出されなかった 677 件からなる．677 件のうち，554 件はいずれの GT でも外部依

© 2026 Information Processing Society of Japan

表 3　計測時の主な環境変数とその値
Table 3　Main environment variables and their values during measurement.

項目	値	値の決まり方
GOOS	linux	GT-imported 系で指定，ツールでは既定値
GOARCH	amd64	実行環境の既定値
CGO_ENABLED	1	実行環境の既定値
GOFLAGS	-mod=mod	スクリプトで指定（原則）
GOTOOLCHAIN	local	スクリプトで指定（原則）

存が検出されず，123 件は GT-all では外部依存が検出された．また，この 123 件のうち 112 件は GT-imported+test でも外部依存が検出された．

5.3　評価対象ツールと実行環境

評価には，Syft v1.46.0[10]，Trivy v0.72.0[11]，cdxgen 12.7.1[12]（Node.js v22.22.2），cyclonedx-gomod v1.10.0[13]（mod モード）を用いた．GT の生成には原則として Go1.26.5 を用いた．計測時の主な環境変数とその値を**表 3** に示す．

各ツールの実行コマンドを**表 4** に示す．<repo>は対象リポジトリのルートディレクトリ，<out>は SBOM の出力先，<limit>は処理を打ち切るまでの秒数を表す．<limit>は通常 300 秒とし，処理時間の長かったリポジトリを再処理する際は 650 秒とした．いずれも CycloneDX JSON 形式で SBOM を生成した．

GT および各ツールによる SBOM は，同一のリポジトリのクローン上で生成した．各環境変数の設定理由と例外処理は，付録 A.3 に示す．比較は依存のモジュールパス単位で行い（name 一致），本編では name 一致を主として，version 一致は付録 A.1 で補う．

5.4　各ツールが依存を読み取る情報源

Syft と Trivy は，リポジトリ内の go.mod と go.sum を読み取って依存を列挙する．コードに記述された import 関係を解析するのではなく，主にこれらのファイルに記録された依存情報に基づいて収集する．go.mod と go.sum には，実際にコードが使う依存だけでなく，テスト用・他 OS 用・開発ツール用の依存も記述されている．そのため，Syft/Trivy は，コードが実際に使う依存より広い範囲を報告する．

cdxgen は，`go list -deps` と `go mod graph` を用いて，コードが実際に使う依存を中心に列挙する．ファイルの記述ではなく，コードから静的に到達可能な依存を収集するため，GT-imported に近い．cyclonedx-gomod（mod モード）は，Go 専用の SBOM 生成ツールである．`go list -m all` で全依存を収集したのち，`go mod why` を用いて，コード上で必要と判断した依存を残す．この絞り込みにより，GT-imported に近い結果となる．

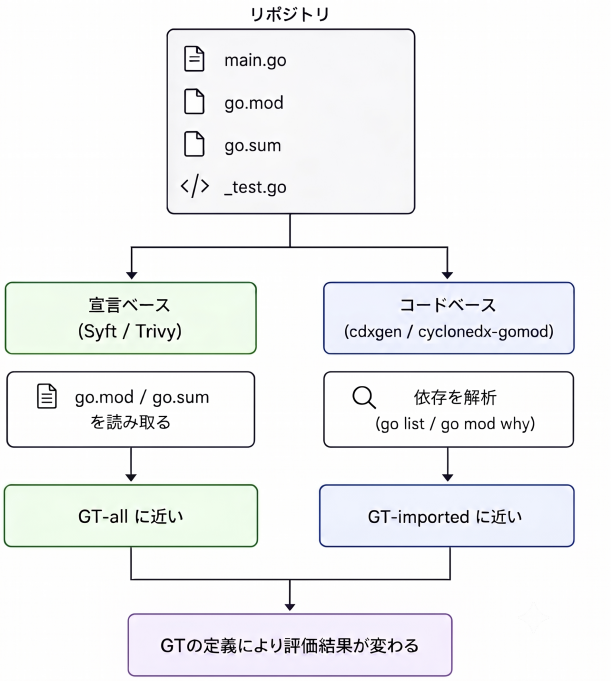


図 3　各ツールの依存取得情報源と最も近い GT の対応関係
Fig. 3　Conceptual relationship between dependency sources and the closest ground truth.

まとめると，Syft/Trivy は「ファイルに記録された依存」を，cdxgen/cyclonedx-gomod は「コードが実際に使う依存」を中心に報告する（**図 3**，**表 5**）．これは各ツールの設計方針の違いであり，どちらかが誤動作しているわけではない．

5.5　評価指標

ツール出力と GT の対応（TP/FP/FN）から，以下の式 (1)〜(3) に示す，precision，recall，F1 を算出する．

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{1}$$

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{2}$$

$$\text{F1} = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \tag{3}$$

precision は，ツールが報告した依存のうち GT に含まれるものの割合であり，過剰報告の少なさを表す．recall は，GT に含まれる依存のうちツールが報告したものの割合であり，見落としの少なさを表す．各 GT 定義に対し，リポジトリごとに precision / recall / F1 を算出し，リポジトリ間で平均する（macro 平均）．各ツールが SBOM を出力できたリポジトリを，そのツールの有効リポジトリとし，SBOM を出力できなかったリポジトリは集計から除外した

© 2026 Information Processing Society of Japan

表 4　各 SBOM 生成ツールの実行コマンド
Table 4　Commands used to execute each SBOM generation tool.

ツール	実行コマンド
Syft	`timeout <limit> syft <repo> -o cyclonedx-json=<out>`
Trivy	`timeout <limit> trivy fs <repo> --format cyclonedx --output <out>`
cdxgen	`timeout <limit> cdxgen -t go <repo> -o <out>`
cyclonedx-gomod	`timeout <limit> cyclonedx-gomod mod -json -output <out> <repo>`

表 5　各 SBOM 生成ツールが読み取る情報源と近い GT
Table 5　Source read by each tool and the closest GT.

ツール	情報源	近い GT
Syft	go.mod, go.sum	GT-all
Trivy	go.mod, go.sum	GT-all
cdxgen	`go list -deps`, `go mod graph`	GT-imported
cyclonedx-gomod	`go list -m all`, `go mod why`	GT-imported

（有効リポジトリ数は 1,493〜1,519 件）．本研究の関心はツール間の順位の変化にあるが，precision と recall の 2 軸のままでは順位を一意に定められず，逆転を示すことができない．そこで，両者の調和平均であり，過剰報告と見落としの双方を考慮した依存集合の総合的な一致度を表す F1 を主指標に用いる．ただし F1 は両者を等しく重み付けするものであり，用途によって重要度は異なるため，precision と recall も併記する．なお，F1 はリポジトリごとに式 (3) で算出したうえで平均しているため，表中の precision と recall の平均値を式 (3) に代入した値とは一致しない．

6. 結果

本章では，3 つの GT 定義に対する各ツールの評価結果，テスト依存の影響，ならびに結果のロバストネスと統計的有意性について述べる．

6.1 GT 定義によるツール優劣の逆転

各ツールの macro 平均 P/R/F1（name 一致）を，3 つの GT 定義ごとに**表 6** に示す．GT-all を正解とすると，go.mod 及び go.sum を広く読む Syft と Trivy は recall が高く（67%程度），コードから静的に到達可能な依存を中心に列挙する cdxgen と cyclonedx-gomod は recall が低い（42〜44%）．このとき F1 は Syft と Trivy が最も高く，cdxgen と cyclonedx-gomod は低い．

一方，GT-imported を正解とすると，全ツールの recall がほぼ等しく高くなる一方で，precision で差がつき，cdxgen と cyclonedx-gomod（91〜92%）が Syft と Trivy（56〜59%）を大きく上回る．順位も入れ替わり，cdxgen と cyclonedx-gomod の F1 が最も高くなる．

すなわち，GT-all では recall，GT-imported では precision という形で，優劣を分ける指標そのものが入れ替わり，どの GT 定義で測るかによってツールの優劣が逆転する（RQ1）．

表 6　各ツールの精度（macro 平均・name 一致，all / imported / imported+test，%）
Table 6　Accuracy per tool (macro, name-match; all / imported / imported+test, %).

ツール	指標	値
Syft	prec.	92.6 / 55.6 / 68.6
	rec.	67.1 / 99.9 / 99.9
	F1	73.0 / 67.0 / 78.3
Trivy	prec.	92.3 / 59.1 / 72.9
	rec.	67.0 / 99.9 / 99.9
	F1	72.5 / 68.4 / 79.7
cdxgen	prec.	94.5 / 91.4 / 92.2
	rec.	42.4 / 99.8 / 82.4
	F1	52.8 / 93.2 / 82.3
cyclonedx -gomod	prec.	99.9 / 92.4 / 92.6
	rec.	43.5 / 99.1 / 81.0
	F1	56.5 / 94.5 / 82.5

6.2 テスト依存の影響

GT-imported+test では，テスト依存を正解に含めることで，Syft と Trivy は recall を高く保ったまま precision が上昇する（例：Syft の precision は imported 時の 55.6%から 68.6%へ上昇）．一方，cdxgen と cyclonedx-gomod はテスト依存を取りこぼすため，recall が低下する（cdxgen は 99.8%から 82.4%へ低下）．このように，テスト依存を GT に含めるか否かも，ツールの評価結果に影響する．

6.3 母数差に対するロバストネス

有効リポジトリ数は，Syft が 1,519 件，Trivy が 1,509 件，cdxgen が 1,493 件，cyclonedx-gomod が 1,501 件であり，ツール間で差がある．この母数差が順位を歪めていないことを確認するため，4 ツールすべてで有効となったリポジトリの集合を共通リポジトリ集合とし，この集合（$n = 1,467$）に限定して再計算した（**表 7**）．値の変化はいずれも 1 ポイント未満であり，ツールの順位も変わらなかった．よって，ツール間の母数差は本研究の結論に影響しない．

© 2026 Information Processing Society of Japan

表 7 母数差のロバストネス検証（imported，name 一致，%）
Table 7 Robustness to differing denominators (imported, name-match, %).

ツール	指標	各自の有効集合	共通集合（$n = 1,467$）
Syft	prec.	55.6	55.9
	rec.	99.9	100.0
	F1	67.0	67.4
Trivy	prec.	59.1	59.6
	rec.	99.9	100.0
	F1	68.4	68.8
cdxgen	prec.	91.4	91.5
	rec.	99.8	99.8
	F1	93.2	93.3
cyclonedx	prec.	92.4	92.7
-gomod	rec.	99.1	99.2
	F1	94.5	94.7

6.4 逆転の統計的有意性

逆転が偶然でないことを確認するため，リポジトリごとの F1 の差について Wilcoxon の符号付き順位検定を行った（共通集合 $n = 1,467$，name 一致）．効果量には matched-pairs rank-biserial correlation を用いた．以下で，分は両者の F1 が一致したリポジトリを表し，中央値差の算出および検定から除外した．GT-imported では cdxgen が Syft に対し 1,280 勝 6 敗 181 分（中央値差 +23.1 ポイント，効果量 0.99），GT-all では逆に Syft が cdxgen に 1,230 勝 57 敗 180 分（中央値差 +18.8 ポイント，効果量 0.95）となり，いずれも $p < 10^{-15}$ であった．すなわち，同じ 2 ツールの勝敗が GT 定義を変えると逆転することが，リポジトリ単位でも統計的に裏付けられた．一方，cdxgen と cyclonedx-gomod の間は GT-imported で効果量 −0.17 と小さく，実質的に互角である．本研究はこの 2 ツールに序列を付けない．以上より，GT 定義の違いによるツール順位の逆転は偶然ではなく，リポジトリ単位でも一貫して観測される現象である．

7. 考察

本章では，GT 定義によってツールの優劣が逆転する理由，実用上の示唆，および本研究の限界について述べる．

7.1 逆転が生じる理由

ツールが読み取る情報源（依存の範囲）と正解 GT の範囲が異なる分だけ，precision または recall が低下する．GT-all は最も広い集合であり，`go.mod` と `go.sum` を広く読む Syft / Trivy は GT-all に含まれる依存を取りこぼしにくいため recall が高い．一方，使う依存を中心に列挙する cdxgen / cyclonedx-gomod は取りこぼしが多く recall が低い．GT-imported は実際に使う依存に絞られるため，コードから静的に到達可能な依存を中心に報告する cdxgen / cyclonedx-gomod は precision が高い．一方，Syft / Trivy はこれより広い範囲を報告するため precision が低い．

よって，同じツール群でも，どの GT で測るかで優劣が逆転する．これは，各ツールがファイルに記録された依存を報告するか，コードが実際に使う依存を報告するかという設計方針の違いに由来する（RQ2）．

7.2 実用化に向けた提言

本研究の結果から，SBOM 生成ツールの用途に応じて，評価のための GT 定義を適切に選択する必要があるといえる．例えば，脆弱性管理の用途においては実際に使う依存を確認したいため，GT-imported が適切であると考えられる．一方で，ライセンス調査の用途においては関わる依存をすべて確認する必要があるため，GT-all が適切であると考えられる．本研究の結果は，SBOM 生成ツールの精度が，正解（GT）をどのように定義するかに依存することを示している．したがって，ツールの精度を報告する際には，正解をどの範囲の依存で定めたかを明示することが望ましい．用途に合わない GT で測ると，ツール選定を誤るおそれがある．

7.3 限界

本研究にはいくつかの限界がある．対象は awesome-go に収録された公開リポジトリに限られ，企業の非公開プロジェクト等への一般化には注意を要する．また，正解 GT を `go list` に基づいて定義しており，この GT の作り方自体が一つの立場であるため，別の GT 定義を採用すれば結果が変わる可能性がある．比較は name 一致（モジュールパス単位）を主としており，バージョンの厳密な一致は付録での補足にとどまる．さらに，各ツールは既定に近い設定で 1 回実行しており，オプションやモードの違いによる感度は検討していない．また，評価対象を GT-imported が空でないプロジェクトに限定したため，GT-imported+test または GT-all では依存が存在する 123 件も除外されている．この選定が比較結果に影響した可能性がある．加えて，1,528 件中 7 件で自モジュールの除外が不完全であったが，macro P/R/F1 への影響は最大 0.1 ポイントであり，ツール順位および主要な結論には影響しなかった．

8. おわりに

本研究では，1,528 件の Go プロジェクトを対象に，依存範囲の異なる 3 つの GT を用いて，4 種類の SBOM 生成ツールの精度を評価した．

RQ1「GT 定義を変えると，SBOM 生成ツールの優劣は変わるか」に対して，GT 定義によってツールの優劣は変わることを示した．例えば，Syft と cdxgen の F1 スコアは，GT-all ではそれぞれ 73.0%と 52.8%であったのに対し，GT-imported ではそれぞれ 67.0%と 93.2%となり，

© 2026 Information Processing Society of Japan

両ツールの優劣が逆転した．また，GT-imported+test では，Syft と Trivy の precision が上昇する一方，cdxgen と cyclonedx-gomod の recall が低下した．

RQ2「優劣が変わるとすれば，その原因は何か」に対して，各ツールが依存を読み取る情報源の違いが原因であることを示した．Syft と Trivy は go.mod および go.sum に記録された広い範囲の依存を報告するのに対し，cdxgen と cyclonedx-gomod はコードから静的に到達可能な依存を中心に報告する．この違いにより，GT-all では Syft と Trivy が，GT-imported では cdxgen と cyclonedx-gomod が高く評価された．

以上より，SBOM 生成ツールの精度は GT 定義に依存するため，評価結果を報告する際には，正解とみなす依存の範囲を明示する必要がある．今後の課題として，対象言語およびデータセットの拡張と，ツールの設定や実行モードを変えた場合の感度分析が挙げられる．

参考文献

[1] S. Yu, et al.: On the Correctness of Metadata-Based SBOM Generation: A Differential Analysis Approach, Proc. DSN, pp.29–36 (2024).

[2] M. Balliu, et al.: Challenges of Producing Software Bill of Materials for Java, IEEE Security & Privacy (2023).

[3] M. F. Rabbi, et al.: SBOM Generation Tools Under Microscope: A Focus on the npm Ecosystem, Proc. SAC, pp.1233–1241 (2024).

[4] L. Zhou, et al.: A Reality Check on SBOM-based Vulnerability Management, Proc. CODASPY (2026).

[5] W. Tang, et al.: Towards Understanding Third-party Library Dependency in C/C++ Ecosystem, Proc. ASE (2022).

[6] F. Reichmann, et al.: Poking Around in the Dark: Why a Shared Understanding of Components Matters, arXiv:2606.02442 (2026).

[7] The Go Authors: Command go (go list), `https://go.dev/cmd/go`

[8] The Go Authors: Go Modules Reference, `https://go.dev/ref/mod`

[9] avelino: awesome-go, `https://github.com/avelino/awesome-go`

[10] Anchore: Syft, `https://github.com/anchore/syft`

[11] Aqua Security: Trivy, `https://github.com/aquasecurity/trivy`

[12] OWASP CycloneDX: cdxgen, `https://github.com/CycloneDX/cdxgen`

[13] OWASP CycloneDX: cyclonedx-gomod, `https://github.com/CycloneDX/cyclonedx-gomod`

[14] The White House: Executive Order 14028, Improving the Nation's Cybersecurity, Federal Register, 86 FR 26633 (2021). `https://www.federalregister.gov/documents/2021/05/17/2021-10460/improving-the-nations-cybersecurity`

[15] 経済産業省：ソフトウェア管理に向けた SBOM（Software Bill of Materials）の導入に関する手引 ver2.0 (2024). `https://www.meti.go.jp/files/900016245.pdf`

© 2026 Information Processing Society of Japan

表 A·1 macro 平均・version 一致（all / imported / imported+test，%）

Table A·1 Accuracy per tool (macro, version-match; all / imported / imported+test, %).

ツール	指標	値
Syft	prec.	91.1 / 54.8 / 67.8
	rec.	66.6 / 99.9 / 99.9
	F1	72.1 / 66.3 / 77.5
Trivy	prec.	90.9 / 58.5 / 72.1
	rec.	66.4 / 99.9 / 99.8
	F1	71.6 / 67.8 / 79.1
cdxgen	prec.	93.5 / 90.8 / 91.6
	rec.	41.9 / 99.7 / 82.3
	F1	52.2 / 92.7 / 81.8
cyclonedx	prec.	99.9 / 92.3 / 92.6
-gomod	rec.	43.5 / 99.1 / 81.0
	F1	56.5 / 94.5 / 82.4

表 A·2 micro 集計・name 一致（all / imported / imported+test，%）

Table A·2 Accuracy per tool (micro, name-match; all / imported / imported+test, %).

ツール	指標	値
Syft	prec.	84.9 / 53.0 / 57.5
	rec.	50.0 / 99.8 / 99.8
	F1	62.9 / 69.2 / 73.0
Trivy	prec.	84.4 / 54.4 / 59.2
	rec.	48.1 / 99.8 / 99.8
	F1	61.3 / 70.4 / 74.3
cdxgen	prec.	88.2 / 82.7 / 83.6
	rec.	33.1 / 99.8 / 92.5
	F1	48.2 / 90.5 / 87.8
cyclonedx	prec.	99.8 / 85.6 / 85.7
-gomod	rec.	36.0 / 98.9 / 91.1
	F1	52.9 / 91.8 / 88.3

付　　録

A.1 version 一致・micro 集計の精度

本文表 6 は macro 平均・name 一致である．参考として，version 一致（**表 A·1**）と，プール合計から算出した micro 集計（**表 A·2**）を示す．いずれも優劣逆転の傾向は保存される．

A.2 TP / FP / FN プール合計（name 一致）

プール合計の TP/FP/FN を**表 A·3** に示す．

A.3 再現のための環境設定

GT-imported と GT-imported+test では，Linux 向けコードの依存を取得するために `GOOS=linux` を指定した．計測環境の OS も Linux であった．GT-all の出力は `GOOS` に依存しない．実験環境における `GOARCH` と `CGO_ENABLED` の実効値は，それぞれ `amd64` と 1 であり，スクリプトでは明示的に指定していない．`GOARCH=amd64` は，x86-64 プロセッサ向けのコードを対象とする設定である．`CGO_ENABLED=1` は，Go のプログラムから C 言語で書かれた処理を利用できるようにする設定である．

`vendor` は，プロジェクトが利用する外部パッケージのコピーを置く，リポジトリ内のフォルダである．本実験では `GOFLAGS=-mod=mod` を設定し，`vendor` 内のコピーを使用せず，go.mod に基づいて依存を調べた．必要な外部パッケージは Go コマンドがダウンロードし，`/tmp/rm_modcache` に保存した．

基本の Go のバージョンには Go1.26.5 を使用し，原則として `GOTOOLCHAIN=local` を設定した．Go toolchain は，go コマンドやコンパイラなど，Go のコードを処理するためのソフトウェア一式である．`GOTOOLCHAIN=local` は，インストール済みの toolchain を使用し，別のバージョンを自動的にダウンロードしない設定である．

Go コマンドが処理の途中で作成するファイルは，`/tmp/rm_gocache` に保存した．また，解析対象リポジトリを一時的に配置するフォルダとして`/tmp/rm_work` を使用した．これらの一時データは GitHub には保存していない．

A.4 GT の生成コマンド

3 つの GT の生成および整形に用いた主要なコマンドを**表 A·4** に示す．実際のスクリプトでは，これらに加えて，処理時間を制限するための `timeout`，出力先の指定，およびエラー出力の制御を行っている．

変数 `gmain` には，`go list -m` によって取得した対象プロジェクト自身のモジュール名を格納する．この変数は，対象プロジェクト自身を依存一覧から除外するために使用する．

`-deps` は，指定した通常コードから静的に到達可能なパッケージと，それらが使用する依存を列挙する．`-test` を追加すると，テストコードから到達可能な依存も列挙対象となる．

GT-imported と GT-imported+test では，`grep` を用いて，モジュール情報が出力されない空行と対象プロジェクト自身を生成時に除外した．その後，`sort -u` によって出力を並べ替え，重複するモジュールを削除した．

GT-all では，`go list -m -e all` の生成結果をそのまま保存し，SBOM との比較時に対象プロジェクト自身と一致するモジュールを除外した．GT-all には Go 標準ライブラリは出力されない．各 SBOM 生成ツールの出力については，対象プロジェクト自身および Go 標準ライブラリが含まれる場合，比較時に除外した．

© 2026 Information Processing Society of Japan

表 A·3　TP / FP / FN プール合計（name 一致）
Table A·3　Pooled TP / FP / FN (name-match).

ツール	all (TP/FP/FN)	imported (TP/FP/FN)	imported+test (TP/FP/FN)
Syft	76,929 / 13,673 / 77,030	47,981 / 42,621 / 120	52,136 / 38,466 / 123
Trivy	71,376 / 13,198 / 77,001	45,970 / 38,604 / 109	50,075 / 34,499 / 112
cdxgen	47,975 / 6,413 / 96,881	45,006 / 9,382 / 77	45,442 / 8,946 / 3,705
cyclonedx -gomod	54,433 / 130 / 96,653	46,700 / 7,863 / 526	46,776 / 7,787 / 4,578

表 A·4　3 つの GT の生成に用いた主要なコマンド
Table A·4　Core commands used to generate the three GTs.

対象	コマンド
事前処理	`gmain=$(go list -m)`

GT-imported	`GOOS=linux go list -deps -e \`
	`  -f '{{with .Module}}{{.Path}} {{.Version}}{{end}}' ./... \`
	`  | grep -v '^$' \| grep -v "^${gmain} \?$" | sort -u`

GT-imported+test	`GOOS=linux go list -deps -test -e \`
	`  -f '{{with .Module}}{{.Path}} {{.Version}}{{end}}' ./... \`
	`  | grep -v '^$' \| grep -v "^${gmain} \?$" | sort -u`

GT-all	`go list -m -e all`

A.5　GT 生成上の注意

以下で述べるエラーや実行条件により，一部のリポジトリでは，実際に生成された GT が本文で定義した包含関係を満たさない可能性がある．

通常の `go list` では，見つからないパッケージや壊れたパッケージがあるとエラーが報告される．`-e` を指定すると，そのようなパッケージがあっても，取得できる情報を用いて処理を継続する．本研究では，一部のパッケージでエラーが発生した場合にも，取得可能な依存を GT に含めるために`-e` を使用した．

ただし，エラーのあるパッケージでは，モジュール名やバージョンを取得できない場合がある．本研究のコマンドは，モジュール情報のない空行を除外するため，そのような依存は GT に含まれない可能性がある．Go 1.26.5 と `GOTOOLCHAIN=local` を用いて最初に GT を生成した際には，パッケージごとのエラーを保存しておらず，欠落した依存を事後的に特定できない．

一つのリポジトリに，それぞれ独自の `go.mod` を持つ複数のフォルダが含まれる場合がある．`go.work` は，それらをまとめて一つの作業対象として扱うための設定ファイルである．`go.work` を使用している状態を workspace mode という．

workspace mode では`-mod=mod` を指定できない．そのため，リポジトリのルートに `go.work` がある場合に限り，`GOFLAGS=-mod=mod` を外して計測した．`go list` 自体を使用できないのではなく，`-mod=mod` との組合せを使用できないためである．

Go 1.26.5 より新しい Go を `go.mod` または `go.work` が要求する場合，`GOTOOLCHAIN=local` では `go list` を実行できない．この場合は `GOTOOLCHAIN=auto` で再計測し，必要な toolchain の自動取得を許可した．ただし，実際に使用された toolchain のバージョンと，再計測対象の完全な一覧は保存していない．

© 2026 Information Processing Society of Japan